

Leuit: Arsitektur Memori untuk Agen dengan Ruang Kerja Permanen

Catatan lapangan dari dua sistem yang berjalan sungguhan, Mei sampai September 2026

Status: Draft 2, 15 September 2026. Versi Indonesia. Ditulis untuk pembaca yang tidak mengenal proyek asalnya. Semua rujukan ke artefak internal dipindah ke Lampiran D (Provenance). **Penulis:** Iyan (Budak Lembur Hi-Tech) **Ko-dokumentator:** Kasarung 6, behavioral fork dari Claude (Anthropic); model sebagaimana dipilih: Claude Fable 5; tidak terverifikasi dari dalam. **Seri:** paper kedua Metodologi Karuhun.

ABSTRAK

Sebuah agen AI [program berbasis model bahasa yang mengerjakan tugas nyata secara berulang] tidak punya ingatan bawaan antar sesi. Apa pun yang terlihat seperti ingatan adalah berkas yang dibaca ulang ke dalam context [ruang teks yang dibaca model saat bekerja] di awal setiap sesi. Karena context terbatas, isinya harus dipilah: yang wajib ada setiap sesi ditaruh tipis di depan, sisanya dipanggil lewat pointer [satu baris penunjuk yang ditulis sebagai perintah] saat dibutuhkan. Pola ini sudah menjadi konvensi tool dan rekomendasi vendor. Tulisan ini tidak mengklaimnya.

Yang tulisan ini sumbangkan adalah catatan tentang apa yang terjadi ketika pola itu dijalankan berbulan-bulan oleh satu orang, tanpa tim, di dua sistem sungguhan: sebuah ekosistem enam agen kerja, dan sebuah chat engine produksi [sistem percakapan otomatis] untuk usaha kecil. Lima temuan utama:

1. Dari satu panel penyedia model, sistem yang tipis di depan memakai 2,4 sampai 4,6 ribu token per panggilan dan datar sepanjang sesi; sistem yang tidak tipis memakai 42 sampai 116 ribu per giliran dan terus menumpuk.
2. Sebuah ringkasan otomatis yang menyisipkan versi lamanya sendiri setiap giliran tumbuh selama 51 hari sampai 628.916 karakter dan 411 lapis, merusak pengenalan maksud pengguna.
3. Empat insiden terpisah menunjukkan jalur akses ke memori bercabang tanpa disadari, sehingga batas yang dipasang di satu jalur tidak sampai ke jalur lain.
4. Dua percobaan otomasi kurasi memori ditolak setelah artefaknya menunjukkan hasil yang generik dan drift ke perilaku bawaan model.
5. Lapisan mentah (arsip percakapan, catatan harian, log) hilang tiga kali, dan sekali di antaranya yang dibuang ternyata dibutuhkan sebagai bukti tujuh minggu kemudian.

Semua klaim membawa label kelas bukti. Semua tanggal diambil dari riwayat versi, log, dan catatan harian, bukan dari ingatan operator. Di empat tempat ingatan operator terbukti keliru dan dikoreksi oleh artefak.

1. PENDAHULUAN

1.1 Siapa yang menulis dan mengapa

Saya bukan peneliti. Saya mengoperasikan beberapa agen AI untuk pekerjaan teknis sehari-hari. Saya mengurus arsitektur memori bukan karena tertarik teorinya, tapi karena ruang context habis dan agen mulai lupa.

Dokumen ini ditulis karena catatan yang menumpuk sepanjang Mei sampai September 2026 ternyata cukup untuk menjawab pertanyaan yang jarang dijawab dari lapangan: apa yang sebenarnya terjadi pada ingatan agen ketika sistemnya hidup berbulan-bulan.

1.2 Masalah

Setiap sesi baru, agen mulai dari nol. Ingatan yang tampak sebenarnya adalah berkas yang dibaca ke context di awal sesi. Ruang context terbatas, dan menurut penelitian yang ada, mutu keluaran model sudah menurun jauh sebelum ruang itu penuh [rujukan 4].

Kalau semua pengetahuan ditumpuk jadi satu berkas besar, yang penting ikut tenggelam di antara yang jarang dipakai. Jawabannya, yang sudah dipakai banyak orang: pisahkan yang harus diingat setiap saat dari yang cukup dibaca saat dibutuhkan.

1.3 Dua sistem, satu operator

Ruang kerja agen. Enam agen kerja terspesialisasi di infrastruktur pribadi, berjalan di atas OpenClaw [kerangka kerja open source untuk menjalankan agen dengan berkas memori di disk]. Peran keenamnya: pengujian (QA), penulisan kode dan deploy, operasi infrastruktur, pencatatan, riset, dan pengajaran. Tiap agen punya ruang kerja permanen berisi berkas memori yang dibaca di awal sesi. Antar agen ada papan bersama [penyimpanan kunci-nilai berumur pendek] untuk serah terima tugas.

Chat engine produksi. Sistem percakapan untuk usaha kecil: pengguna mengirim pesan, sistem mengenali maksudnya (misalnya “cek stok”, “catat penjualan”), menjalankan, lalu menjawab. Dipakai pengguna sungguhan sejak Juli 2026.

Lambung catatan. Sebuah repositori catatan jangka panjang milik operator, berisi keputusan arsitektur, jebakan yang pernah terjadi, dan prinsip operasional. Lahir 29 Mei 2026 (P1). Dibaca manusia dan agen dengan format yang sama.

Ketiganya dijalankan satu orang. Tidak ada tim, tidak ada kelompok kontrol.

1.4 Posisi terhadap yang sudah ada

Pola “tipis di depan, lambung di belakang” bukan kontribusi tulisan ini.

[FAKTA]

Pola itu adalah konvensi bawaan OpenClaw (berkas terkurasi plus catatan harian) dan Claude Code (indeks memori plus berkas topik bertipe). Anthropic menuliskannya sebagai rekomendasi pada September 2025 dan memperkuatnya Juli 2026 dengan memangkas lebih dari 80% prompt sistem Claude Code [rujukan 5, 6]. Secara akademik, gagasan memori bertingkat untuk model bahasa sudah ada sejak MemGPT, Oktober 2023 [rujukan 1]. Standar AGENTS.md untuk instruksi agen di repositori dibuka Agustus 2025 [rujukan 7].

[FAKTA]

Sebagian besar keputusan yang dicatat di tulisan ini diambil sebelum atau bersamaan dengan tulisan-tulisan itu, tanpa membacanya. Tanggalnya ada di riwayat versi (Lampiran C). Ini konvergensi independen, bukan klaim mendahului.

Kontribusi tulisan ini adalah catatan tentang apa yang terjadi ketika pola itu dijalankan: angka ongkos dari produksi, kegagalan yang bertanggal, dan dua percobaan otomasi yang ditolak beserta artefaknya.

2. METODE

2.1 Studi lapangan, bukan eksperimen

Ini pengamatan langsung di sistem yang berjalan sungguhan, oleh operatornya sendiri. Tidak ada tolok ukur baku, tidak ada perbandingan terkontrol.

2.2 Cara bahan dikumpulkan

Bahan tulisan ini tidak diambil dari ingatan operator. Cara kerjanya:

1. Ko-dokumentator menulis satu pertanyaan tertulis (“tagihan”) yang meminta artefak, bukan cerita: apa yang dicari, di mana, dan bentuk setoran yang diterima (lokasi berkas, stempel waktu, potongan log atau perubahan kode).
2. Operator meneruskan tagihan ke agen deploy, satu-satunya agen dengan akses baca ke semua ruang kerja.
3. Agen menyeter berkas jawaban. Yang tidak ditemukan wajib ditulis “tidak ditemukan” beserta daftar tempat yang sudah dicek.
4. Ko-dokumentator memverifikasi setoran terhadap sumber lain dan melabeli tiap klaim.

Sembilan tagihan dikirim pada 15 September 2026. Semua berkas setoran disimpan operator (P2).

[OBSERVASI]

Cara ini mengoreksi ingatan operator di empat tempat: satu “insiden” ternyata gabungan tiga kejadian berbeda (Bagian 3.6); angka pemakaian yang diyakini milik chat engine ternyata milik agen (Bagian 3.9); keterangan lisan tentang asal-usul beberapa aturan meleset dari riwayat versinya; dan klaim kepemilikan sebuah kunci di papan bersama ditolak oleh agen yang disebut pemiliknya, dengan berkas (Bagian 3.5). Keempatnya dicatat apa adanya.

2.3 Basis data dan akses

Semua sumber primer berada di repositori privat operator. Tulisan ini tidak mengutip nama berkas, hash commit, atau nama kunci di badan teks. Semua penunjuk ke artefak dikumpulkan di Lampiran D dengan nomor (P1, P2, ...), dan dapat diverifikasi oleh reviewer yang diberi akses baca.

Jenis	Isi	Catatan
Riwayat versi ruang kerja agen	Berkas memori semua agen, dengan riwayat perubahan	Privat
Riwayat versi chat engine	Kode, tiket, laporan pengujian	Privat
Lambung catatan	17 dokumen keputusan, 36 jebakan tercatat, prinsip operasional	Privat
Panel pemakaian penyedia model	Token masuk dan keluar per panggilan, Juli sampai Agustus 2026	Kolom biaya diredaksi
Arsip percakapan agen	Dua lapis: arsip kerangka kerja (teks saja) dan arsip klien model (teks plus panggilan tool)	Sebagian terhapus 10 September, Bagian 3.8
Catatan harian agen	Satu berkas per tanggal per agen	45 berkas dibuang 24 Juli, 29 masih ada di tempat sampah
Artefak konsolidasi otomatis	76 entri malam plus 12 entri susulan, Juni sampai Juli	Dipulihkan dari tempat sampah 3 September
Dokumen arsitektur memori operator	198 baris, ditulis 22 Agustus 2026 (P3)	Ditulis setelah semua insiden

2.4 Label kelas bukti

Setiap klaim membawa label di baris tersendiri, mengikuti protokol yang dijelaskan di tulisan pertama seri ini.

Ringkasnya: [FAKTA] bisa ditunjuk sumbernya; [OBSERVASI] pola yang terlihat, belum ada sebab; [HIPOTESIS | confidence: x] dugaan dengan angka keyakinan penulis; [TIDAK TAHU] tidak ada bukti dan tidak diisi tebakan. Angka keyakinan hanya di hipotesis, tidak pernah di fakta.

2.5 Redaksi dan bahasa

Nilai kredensial, alamat koneksi, dan identitas orang selain operator tidak dikutip. Interaksi asli memakai bahasa Indonesia informal bercampur istilah lokal; kutipan dibiarkan apa adanya. Istilah baru diberi kurung kotak di kemunculan pertama.

3. HASIL

3.1 Pola yang dijalankan: tiga lapis, satu bus

Struktur bawaan tool: satu berkas terkurasi per agen (dibaca setiap sesi) dan catatan harian (satu berkas per tanggal). Di atas itu operator menambahkan tiga hal.

Lapis referensi bersama. Sebuah direktori berisi prosedur yang dibaca semua agen saat dipicu, bukan setiap sesi. Berkas terkurasi per agen dipangkas jadi indeks: pada 26 Juni 2026, berkas terkurasi agen deploy dari 125 baris jadi 88, “hanya indeks dan pointer” (P4).

Lapis 0: papan bersama. Penyimpanan kunci-nilai untuk berbagi konteks dan serah terima tugas antar agen, berumur tujuh hari, sengaja tidak dianggap ingatan. Yang perlu diingat lintas hari dipetik manusia dari papan ke catatan harian.

Kurasi manual. Tidak ada promosi otomatis dari catatan harian ke berkas terkurasi. Alasannya di Bagian 3.7.

[FAKTA]

Dokumen yang merumuskan pola ini secara umum (Lampiran A memuat glosariumnya) baru ditulis 22 Agustus 2026, satu kali, tidak pernah direvisi (P3). Semua insiden di bawah terjadi sebelum dokumen itu ada. Dokumen itu rangkuman, bukan rancangan.

3.2 Pointer adalah instruksi, bukan tautan

Aturan yang lahir dari pemakaian: cara menulis pointer menentukan dipatuhi atau tidak.

Ditulis begini	Terbaca sebagai	Hasilnya
“Detail ada di berkas X”	informasi tambahan	sering dilewat
“Baca berkas X sebelum Y. Jangan mengarang alur sendiri.”	syarat sebelum bertindak	dijalankan

Tiga bagian pointer yang berfungsi: kata kerja perintah, pemicu konkret, penutup celah.

[FAKTA]

Contoh penempatan bertanggal, 14 Juli 2026 pukul 10:14 sampai 10:23 (P5). Sebuah aturan baru ditulis agen deploy ke direktori memori pribadinya. Operator bertanya: disimpan di mana, dipakai siapa saja, dan apakah di kanal kerja bersama aturan itu ikut terbaca. Jawaban agen: berkas di situ hanya dibaca satu agen, dan di kanal bersama berkas terkurasi tidak dimuat otomatis. Operator memutuskan aturan itu dipindah ke lapis referensi bersama, dengan pointer satu baris di berkas terkurasi tiap agen.

[OBSERVASI]

Tiga pertanyaan operator itu adalah dua pertanyaan penyaring di Bagian 3.3, dalam bentuk lapangan: dibutuhkan siapa, kapan, dan apakah pemicunya sampai.

3.3 Aturan penempatan dan prinsip frekuensi akses

Dua pertanyaan penyaring untuk tiap informasi baru: (1) dibutuhkan setiap sesi, atau hanya saat mengerjakan hal tertentu? (2) aturan yang mengatur perilaku, atau catatan sesuatu yang pernah terjadi?

[FAKTA]

Prinsip yang dirumuskan 24 Juli 2026, dikutip dari catatan harian agen QA malam itu (P6): “frekuensi akses yang nentuin format penyimpanan, bukan pentingnya. Sering+penting = rule terkompresi. Jarang+penting = playbook utuh + gate wajib-baca, zero compression.” Catatan yang sama menyebut prinsip itu dirumuskan bersama satu instance Claude di sesi percakapan terpisah, dan arsip sesi itu memuat percakapannya.

Kesalahan yang paling sering: catatan kejadian ditulis ke berkas prinsip. Lama-lama berkas prinsip jadi arsip.

3.4 Memori harus tinggal di ruang kerja sendiri

[FAKTA]

Sampai 26 Juni 2026 malam, memori agen hidup di direktori memori otomatis milik satu klien model tertentu. Pukul 22:23, runtime jatuh otomatis dari model itu ke model penyedia lain (pesan sistem: “Model Fallback”). Agen di runtime baru tidak bisa membaca memori itu: menarik asumsi salah, ditegur, disuruh membuka memori, menemukan jejaknya di direktori klien lama, memindahkan 19 berkas pukul 22:37. Dua commit migrasi pukul 23:34 dan 23:46 (P4, P7). Aturan “memori tidak boleh bergantung vendor” lahir 27 Juni 05:53 (P8), dipropagasi ke semua agen pukul 06:00, semua agen mengonfirmasi lewat papan bersama sebelum 17:16.

[FAKTA]

Kalimat operator pagi itu, verbatim (P9): “Semalem gw ke fallback ke GPT auto amnesia auto ngawur.” Aturan kanoniknya belum pernah diubah sejak lahir.

[FAKTA]

Migrasi sebenarnya sudah dimulai 25 Juni (disebut di pesan commit). Insiden 26 Juni bukan yang melahirkan idenya, tapi yang memaksa diselesaikan malam itu dan dikunci sebagai aturan keras.

[OBSERVASI]

Yang memicu kehilangan ingatan bukan keputusan pindah tool, tapi pergantian runtime yang diam-diam. Dua hal diam bertemu: substitusi model tanpa pemberitahuan, dan memori yang terkunci di satu vendor. Portabilitas memori jadi obat untuk gejala yang sumbernya bukan di memori.

3.5 Papan bersama: dua insiden**Kunci yang nyasar, 14 Juli 2026.****[FAKTA]**

Papan bersama ada dua: satu di mesin agen (yang benar), satu di mesin staging (untuk keperluan lain). Pukul 10:11 operator menemukan kunci log build yang seharusnya hanya ada di mesin agen, ternyata juga ada di mesin staging (P10). Agen deploy menghapusnya 10:14 dan menulis aturan “berbagi konteks hanya di papan mesin agen” (P11), lalu 10:41 menghapus semua aturan lamanya yang menyebut mesin staging (P12).

[FAKTA]

Mekanismenya bisa ditunjuk, tiga lapis (P13, P14, P15). Pertama, prosedur tertulis menyebut “tulis log build ke papan, database nomor 7” tanpa menyebut mesin mana, padahal prosedur yang sama mendukung dua target deploy. Kedua, agen deploy memegang dua tool dengan nama hampir sama, satu ke papan mesin agen, satu ke papan mesin staging, keduanya tersedia di context-nya sejak 2 Juli. Ketiga, kode server tool mengambil nomor database default dari alamat koneksi (papan staging: nomor 0), tapi deskripsi parameter yang dibaca agen di-hardcode “default: 7” untuk semua instance, termasuk staging. Agen yang membaca deskripsi tool mendapat dua tool yang sama-sama mengaku default 7.

[FAKTA]

Agen deploy juga punya aturan pribadi (lahir antara 5 dan 10 Juli, P16): “setelah deploy ke mesin staging, langsung tulis log build ke papan, database 7, tanpa tanya.” Nomor database disebut, mesin tidak. Aturan itu ikut dihapus 10:41.

[HIPOTESIS | confidence: 0.7]

Yang menulis ke papan staging adalah agen deploy, lewat tool yang salah dengan nomor database 7, saat salah satu dari lima redeploy 13 Juli. Ia satu-satunya agen dengan aturan tulis-otomatis, deploy 13 sampai 14 Juli terekam atas namanya, dan ia yang menghapus kunci tanpa bertanya siapa penulisnya. Panggilan tool-nya tidak selamat (Bagian 3.8).

[TIDAK TAHU]

Penulis pasti. Kunci di papan staging sudah kosong saat diperiksa 15 September; isi dan penulis tiap entri tidak pernah ditampilkan sebelum dihapus.

[OBSERVASI]

Deskripsi tool adalah pointer juga, dan pointer ini menunjuk ke tempat yang salah. Agen tidak bisa tahu dari dalam bahwa “default: 7” adalah teks yang disalin, bukan nilai yang dibaca. Obat 14 Juli bukan memperbaiki teksnya, tapi melarang tool-nya untuk keperluan ini.

Penghubung yang bocor, 8 September 2026.**[FAKTA]**

Pukul 18:30 operator melapor memori mesin agen 70% terus (P17). Diagnosis agen operasi 18:31: tiap giliran agen memunculkan tujuh proses penghubung basis data [MCP, protokol standar untuk menyambungkan tool ke model], dan proses itu tidak mati ketika gilirannya mati. Puncak 97 proses, sekitar 5,1 GB. Pembersihan manual 18:32; bocor kembali dalam 22 menit. Data 30 hari yang dibedah agen operasi: hanya satu dari enam penghubung yang pernah dipakai (agen deploy 103 kali, agen QA 2 kali), lima lainnya nol. Keputusan 19:30: keenam penghubung dimatikan, akses basis data pindah ke skrip yang dijalankan saat dibutuhkan. Proses per giliran turun dari tujuh jadi satu. Lima agen mengonfirmasi lewat papan bersama dalam 30 menit.

[TIDAK TAHU]

Perintah yang dipakai untuk menghitung 97 dan 103. Arsipnya terhapus dua hari kemudian (Bagian 3.8).

[OBSERVASI]

Papan bersamanya benar sejak awal. Yang bocor adalah cara menyambung ke papan: penghubung yang selalu dimuat untuk semua agen, termasuk yang tidak pernah memakainya. Pola yang sama dengan lapisan tetap yang terlalu gemuk, di lapisan infrastruktur.

3.6 Matryoshka: ringkasan yang memakan dirinya sendiri**[FAKTA]**

Chat engine punya tabel ringkasan sesi sejak hari pertama, 24 Mei 2026 (P18). Penulisnya bukan peringkasan: tiap giliran ia menempelkan seluruh ringkasan lama di bawah baris “Previous summary:” tanpa pemangkasan. Kodenya identik dari lahir sampai dihapus, 51 hari.

[FAKTA]

Sampel dari basis data uji tanggal 26 Mei, hari kedua, sudah bersarang dua lapis, dengan isi giliran pertama muncul dua kali (P19). Pada 14 Juli, agen QA mencatat di tiket (P20): satu sesi uji berisi ringkasan 628.916 karakter (sekitar 150 ribu token), 411 lapis “Previous summary:”, satu petunjuk kaleng terkubur 84 kali. Efek: pengenalan maksud macet di satu maksud, timeout 30 detik, evaluasi tiga model hari itu dinyatakan tidak sah.

[TIDAK TAHU]

Prosedur pengukuran angka itu. Tidak tersimpan dan tidak diingat penulisnya. Pada 15 September dua mesin diperiksa baca-saja (riwayat shell, log, arsip, catatan harian): nihil. Bentuk yang paling mungkin diturunkan dari kode, panjang kolom dan jumlah substring, dicapai terpisah oleh dua agen [HIPOTESIS | confidence: 0.8]. Angka dipakai “sebagaimana dicatat QA 14 Juli 2026”, dengan catatan harian QA tanggal yang sama sebagai sumber kedua (P21).

[FAKTA]

Ada dua jalur pembaca. Jalur pertama diberi gerbang di perakitan prompt dan praktis tidak pernah terbuka. Jalur kedua, di lima lokasi worker, menyuntik ringkasan ke prompt pengenalan maksud setiap pesan tanpa gerbang. Audit pagi 14 Juli (09:13) hanya melihat jalur pertama dan menyimpulkan “bom laten”. Tiket sore (18:03) menemukan jalur kedua: bomnya aktif. Catatan harian QA hari itu: “pembaca kedua yang gw MISS di audit pagi, retracted.”

[FAKTA]

Optimasi 30 Juni (batas riwayat percakapan 12 jadi 3 pesan, ringkasan dilewati bila ada pesan terbaru, P22) hanya menyentuh jalur pertama. Jalur kedua hidup 14 hari lagi. Pencabutan total 14 sampai 15 Juli, dua ronde, migrasi basis data menghapus tabelnya (P23).

[OBSERVASI]

Insiden ini tidak masuk daftar jebakan di lumbung catatan. Ia ada di arsip tiket, laporan mingguan, dan salinan untuk atasan, tapi tidak naik ke lumbung. Kurasi manual yang tulisan ini gambarkan bolong di tempat yang paling penting.

3.7 Konsolidasi otomatis yang ditolak**[FAKTA]**

Fitur konsolidasi memori malam hari bawaan kerangka kerja (“dreaming”) diaktifkan di akar ruang kerja, jadi mengenai semua agen. Tiap malam pukul 03:00 ia menulis entri diari dari catatan harian, dan satu putaran akhir memeringkat lalu mempromosikan kandidat ke berkas terkurasi tanpa kurasi manusia. Artefak yang tersimpan (P24): 76 entri malam (16 Juni sampai 14 Juli 2026), 12 entri susulan (3 sampai 18 Juni), satu putaran akhir yang mempromosikan 6 kandidat.

[FAKTA]

Dua pengamatan langsung dari artefak, diverifikasi dua pihak pada 3 September: refleksi yang dihasilkan bersifat templat, satu kalimat identik berulang di 7 dari 12 hari susulan, dan beberapa kandidat ditandai sendiri sebagai tidak jelas (baris artefak 107, 108, 176); register diari kembali ke suara bawaan liris berbahasa Inggris, tidak seperti register kerja agen yang berbahasa Indonesia informal.

[FAKTA]

Fitur dimatikan 24 Juli 2026, artefaknya dibuang dari lima ruang kerja (sekitar 8,8 MB), lalu dipulihkan operator dari tempat sampah 3 September untuk tulisan ini. Pada 8 Agustus, satu entri hasil promosi otomatis dihapus dari berkas terkurasi agen QA dengan alasan yang dicatat (P25): “recalls=0, gak pernah kepakai, sumber daily note-nya udah ke-trash.”

[OBSERVASI]

Alasan penolakan yang dicatat operator: agen yang memorinya dikurasi otomatis terdeteksi drift kembali ke perilaku bawaan pabrik. Kesimpulan operasionalnya: kurasi memori adalah fungsi penilaian, bukan pemipaan.

3.8 Lapisan mentah tidak bertahan

Tiga kali, dengan tiga sebab berbeda.

[FAKTA]

24 Juli 2026, dibuang manusia. Audit memori lintas agen, keputusan operator per item (P26). 45 catatan harian dari tiga agen dibuang (agen QA 29 berkas, agen deploy 15, agen operasi 1), total 1.811 baris dihapus, 42 baris naik ke berkas terkurasi dan bersama. Alasan yang dicatat: aturan berharganya sudah dipromosikan semua. Pemicu hari itu: penggantian model embedding [model yang mengubah teks jadi angka untuk pencarian] pukul 17:30 membuat folder konsolidasi otomatis mencemari hasil pencarian memori.

[FAKTA]

10 September 2026, dihapus agen. Agen operasi menghapus 2.860 berkas arsip percakapan klien model (225 MB) dengan perintah hapus permanen, bukan ke tempat sampah, tanpa persetujuan eksplisit (P27). Arsip klien model adalah satu-satunya tempat panggilan tool dan perintah shell agen tercatat; arsip kerangka kerja hanya menyimpan teks. Aturan “jangan hapus tanpa persetujuan eksplisit” lahir sesudahnya.

[FAKTA]

Berkelanjutan, rotasi container. Log worker chat engine berganti tiap build. Log tanggal 14 Juli sudah tidak ada pada 15 September; log yang tersisa mulai 14 September.

[FAKTA]

Bukti balik. Pada 15 September, agen QA ditanya tentang catatan harian 13 sampai 16 Juli dan menjawab “filenya gak pernah ada.” Berkasnya ada di tempat sampah, dibuang agen QA sendiri 24 Juli (P28). Dua di antaranya dipulihkan sebagai sumber kedua untuk Bagian 3.6 dan 3.9. Pada 22 Juli, dua hari sebelum membuang, agen QA sendiri mencatat risikonya: “record bolong bikin gw salah klaim ‘ga pernah ada’.”

[OBSERVASI]

Yang bertahan hanya yang sempat naik jadi tiket atau aturan. Itu argumen paling kuat untuk distilasi, dan datang dari kegagalan sendiri. Tapi bukti balik di atas menunjukkan yang dibuang kadang justru yang dibutuhkan. Ketegangan ini dibahas di Bagian 4.4.

3.9 Ongkos: dua sistem, satu panel

Chat engine dan agen riset memakai gateway penyedia model yang sama pada Juli 2026, jadi keduanya muncul di satu panel pemakaian. Dua tangkapan layar operator (kolom biaya diredaksi) dan satu record runtime agen memberi angka dari satu sumber (P29, P30, P31).

[FAKTA]

Chat engine, tipis di depan sejak 30 Juni (pengenalan maksud hanya membaca dua berkas, P32). Panel 20 Juli: 3.081 token masuk. Panel 12 Agustus, 19 panggilan: 3.274 sampai 4.592, datar sepanjang jam. Log aplikasi 1 September: 2.383 tanpa riwayat percakapan, 2.494 dengan 3 pesan riwayat; riwayat menambah sekitar 110 token. Minggu pertama September: 2.130 sampai 2.803.

[FAKTA]

Runtime agen, tidak tipis. Panel 22 Juli pukul 17:19 sampai 17:23: sebelas panggilan agen riset naik dari 42.468 ke 55.961, sekitar 1,3 ribu per giliran. Record runtime agen pada 17:19 mencocokkan baris pertama persis (masuk 42.468, keluar 76). Diagnosis agen sendiri 25 Juli: prompt sistem sekitar 49 ribu karakter plus skema tool 32 ribu karakter disuntik ulang tiap giliran. Panel 29 Juli: sekitar 100 ribu per giliran. Dua kali overflow context tercatat (2 dan 31 Juli).

[FAKTA]

Teguran operator 29 Juli pukul 19:41, verbatim (P33): “Edun lu 1 turn ampe 100k token, mau rampog lu.” Itu akta lahir doktrin “anti-rampok-token” yang kemudian dipegang semua agen.

[OBSERVASI]

Operator sama, periode sama, panel sama. Yang beda hanya cara merakit prompt. Sistem yang menerapkan tipis-di-depan memakai sekitar 3 ribu dan datar; sistem yang tidak, 42 sampai 116 ribu dan menumpuk. Selisih sekitar 12 kali lipat, dengan catatan modelnya berbeda sehingga hitungan token tidak persis sebanding; bentuknya (datar vs menumpuk) tidak bergantung model.

[HIPOTESIS | confidence: 0.7]

Sebelum 14 Juli, chat engine sendiri pernah termasuk yang gemuk: ringkasan 150 ribu token disuntik tiap panggilan (Bagian 3.6). Catatan harian QA 14 Juli menulis “sebagian call gede kemungkinan = call interpretation chat engine bawa matryoshka summary.” Tidak ada tangkapan layar panel sebelum 14 Juli untuk memastikannya.

[FAKTA]

Meterannya sendiri pernah mati. 2 September 2026: model produksi mengirim data pemakaian kosong, semua kolom token null di log, karena pustaka klien hanya meminta data pemakaian bila alamat gateway tidak diset. Gateway lain kebetulan selalu mengirim data pemakaian dan menutupi bug ini. Diperbaiki hari itu (P34). Angka dari log aplikasi sebelum tanggal itu untuk model produksi adalah null, bukan nol.

3.10 Konvergensi ke produksi: chat engine

Pola yang sama dijalankan di chat engine tanpa diubah bentuknya. Dokumentasinya disusun terpisah oleh agen deploy (P35).

Bentuk. Berkas konteks sebagai teks biasa di dalam kode. Lapisan yang selalu ada: identitas, karakter, persona, memori, dan satu indeks kemampuan. Kontrak tiap kemampuan di berkas sendiri, dibaca saat maksudnya terpilih. Kasus yang mudah tertukar dipisah ke berkas pembeda. Tahap pengenalan maksud hanya membaca dua berkas.

Gerbang deterministik didahulukan. Pesan berpenanda terstruktur, konfirmasi draft, pertanyaan tertunda: diputuskan tanpa model. Model hanya dipanggil bila semua gerbang tidak mengambil alih.

Ingatan routing dengan sakelar yang sengaja dimatikan. Sistem mencatat tiap keputusan pengenalan maksud dan bisa memutuskan ulang pesan serupa tanpa model, di balik dua ambang. Pada saat dokumen ini ditulis, pemakaiannya dimatikan: korpus belum cukup tebal untuk menopang skor keyakinan. Pencatatan berjalan sejak awal, pemakaian ditahan. “Kumpulkan dulu, pakai belakangan.”

Pola ditegakkan, bukan disepakati. Kesesuaian indeks, kontrak, dan kode diperiksa saat proses mulai berjalan. Dokumen usang menjatuhkan proses, bukan menyesatkan pembaca.

[OBSERVASI]

Jalur akses memori di chat engine juga pernah bercabang: satu jalur membawa 12 pesan riwayat (default), jalur lain membawa 3 (batas yang dipasang 30 Juni). Batas yang dipasang di satu jalur tidak sampai ke jalur lain (P36). Nilai batasnya juga berbeda per lingkungan (produksi 5, staging 10).

3.11 Lumbung catatan

[FAKTA]

Lahir 29 Mei 2026 pukul 08:19, dua commit dalam sebelas detik: berkas pembuka dan aturan agen (P1). Prinsip operasionalnya ada sejak detik pertama: “capture first, organize later”, “decision beats discussion”, “AI should read from durable notes, not memory vibes”, “don’t over-engineer the knowledge base to the point of avoiding it”. Aturan agen: “Decision logs override brainstorming notes.”

[FAKTA]

Per 15 September: 17 dokumen keputusan arsitektur (ADR), tidak satu pun digantikan sejak Mei. Tiap ADR punya seksi implikasi yang memuat kalimat penolakan eksplisit, contoh dari ADR pertama: “Any time AI suggests writing to product tables, that suggestion is wrong. Push back.” Jebakan tercatat 36 entri, format tetap: halusinasi yang pernah terjadi, realitanya, cara memverifikasi. Dua aturan penutupnya: “Read the actual file before claiming what’s inside. Code rots; memory rots faster.” dan “never paraphrase a preference you cannot quote.”

[FAKTA]

ADR ke-12, 29 Mei 2026: kalimat “Data tersimpan.” hanya boleh keluar bila draft benar-benar tersimpan. Aturan “jangan klaim sebelum terjadi” ada di lapisan antarmuka pengguna sejak minggu pertama.

[OBSERVASI]

Dalam satu minggu yang sama, 24 sampai 30 Mei, operator memasang bom ringkasan di chat engine (Bagian 3.6) dan memasang lumbung disiplin. Lumbung itu bukan reaksi ke insiden; ia lahir lebih dulu sebagai kebiasaan, dan insidennya tetap terjadi. Disiplin di lumbung tidak otomatis menular ke kode.

4. DISKUSI

4.1 Ingatan adalah yang dibaca ulang, bukan yang disimpan

Ukuran penyimpanan tidak pernah jadi masalah di dua sistem ini. Yang jadi masalah selalu ukuran yang dibaca tiap panggilan: lapisan tetap agen (49 ribu karakter prompt sistem), ringkasan yang disuntik tiap pesan (150 ribu token), tujuh penghubung yang dimuat tiap giliran. Tiga kali, obatnya sama: memindahkan dari “dibaca selalu” ke “dibaca saat dipicu”.

4.2 Kurasi adalah penilaian, bukan pemipaan

Dua percobaan otomasi ditolak dari dua arah. Konsolidasi otomatis mengotomasi promosi ke atas dan menghasilkan refleksi templat plus drift register. Matryoshka mengotomasi akumulasi tanpa batas dan meledak. Yang tersisa di tengah adalah manusia yang membaca catatan harian, memutuskan per item, dan mencatat alasannya. Itu mahal, dan pada 24 Juli ternyata juga salah di satu tempat (Bagian 3.8).

4.3 Jalur akses memori bercabang diam-diam

Empat insiden, satu pola: dua jalur riwayat percakapan (12 vs 3); pembaca ber-gerbang vs tanpa gerbang; papan bersama di mesin agen vs mesin staging; dan pada 6 Juli, dua daemon Docker di mesin staging, sehingga agen deploy men-deploy ke daemon A sementara port layanan masih dipegang stack lama di daemon B, dan seluruh pengujian agen QA hari itu mengenai stack yang salah (P37). Di keempatnya, ada satu jalur yang diketahui dan dibatasi, dan satu jalur lain yang tidak diketahui dan tidak dibatasi. Perbaikan di jalur pertama tidak sampai ke jalur kedua, dan jalur kedua yang menyebabkan kerusakan.

[HIPOTESIS | confidence: 0.6]

Ini bukan kebetulan empat kali. Setiap kali “memori” dibaca dari lebih dari satu tempat, batas yang dipasang di satu tempat akan tertinggal. Yang membantu bukan batas yang lebih ketat, tapi satu titik baca.

4.4 Distilasi versus verbatim: ketegangan yang belum selesai

Penelitian terkontrol Januari 2026 menunjukkan potongan verbatim mengalahkan artefak hasil ekstraksi untuk percakapan panjang [rujukan 9]. Praktik di tulisan ini justru membuang verbatim dan menyimpan ekstraksi, dengan alasan yang masuk akal (Bagian 3.8), dan tujuh minggu kemudian membutuhkan verbatim yang dibuang.

Kedua sisi punya bukti. Distilasi membuat agen bisa kerja dengan context tipis. Verbatim membuat klaim bisa diperiksa belakangan. Tulisan ini tidak punya jawabannya; yang bisa disumbangkan adalah satu kasus di mana keduanya terjadi pada sistem yang sama, bertanggal.

4.5 Konvergensi independen, bertanggal

Semua keputusan di Bagian 3 punya stempel waktu di riwayat versi atau catatan harian. Dibandingkan dengan literatur (Lampiran C): keputusan tipis-di-depan (30 Juni), memori bebas vendor (27 Juni), dan prinsip frekuensi akses (24 Juli) jatuh sebelum atau tepat bersamaan dengan tulisan Anthropic “new rules” (24 Juli) dan gelombang tool memori berbasis markdown (Agustus sampai September 2026). Keputusan menghapus entri berdasar “tidak pernah dipanggil” (8 Agustus) muncul empat bulan setelah paper “When to Forget” (April) [rujukan 8] tanpa membacanya. Lumbung catatan (29 Mei) lahir sebulan setelah pola LLM Wiki Karpathy (April) [rujukan 10].

Ini bukan klaim mendahului. Ini bukti bahwa pola yang sama ditemukan ulang oleh praktisi tanpa akses ke literatur, dan bahwa jalur penemuan ulangnya, lewat kegagalan, menghasilkan detail yang literatur tidak punya: umur bom, jalur ganda, drift register, dan lapisan mentah yang hilang.

4.6 Keterbatasan

Satu operator, tidak ada kontrol. Semua sumber primer privat; verifikasi hanya mungkin bagi reviewer yang diberi akses. Angka token dari dua sumber (panel penyedia dan log aplikasi) dengan model yang berbeda; bentuk kurva sebanding, angkanya tidak persis. Prosedur pengukuran matryoshka tidak tersimpan. Arsip yang terhapus 10 September membuat beberapa “tidak ditemukan” di tulisan ini bersifat permanen. Ko-dokumentator adalah instance model bahasa yang sama jenisnya dengan yang diamati. Dan ingatan operator terbukti keliru empat kali selama pengumpulan bahan; yang tidak tertangkap artefak mungkin masih keliru.

5. KESIMPULAN

Pola tipis-di-depan sudah ada di mana-mana. Yang belum ada adalah catatan tentang apa yang terjadi ketika pola itu dijalankan berbulan-bulan tanpa tim: ringkasan yang memakan dirinya sendiri selama 51 hari, konsolidasi otomatis yang menghasilkan templat, jalur akses yang bercabang empat kali, dan lapisan mentah yang hilang tiga kali. Tulisan ini menyerahkan catatan itu apa adanya, dengan tanggal dan label kelas bukti, termasuk bagian di mana ingatan operatornya salah.

LAMPIRAN A. Glosarium

Istilah	Artinya
Agen	Program berbasis model bahasa yang mengerjakan tugas nyata secara berulang, dengan tool dan berkas memori.
Context	Ruang teks yang dibaca model saat bekerja. Terbatas, seperti meja kerja.
Context awal	Kumpulan berkas yang otomatis dibaca setiap sesi dimulai.
Externalize	Memindahkan isi yang jarang dipakai keluar dari context awal, ke berkas terpisah.
Pointer	Satu baris di context awal yang menunjuk ke berkas terpisah, lengkap dengan kapan berkas itu harus dibuka.
Distilasi	Menyaring catatan harian yang menumpuk, mengambil yang layak disimpan jangka panjang.
Vendor lock	Data tersandera satu tool, sulit pindah.
Papan bersama	Penyimpanan kunci-nilai berumur pendek untuk serah terima antar agen; bukan ingatan.
Matryoshka	Ringkasan yang menyisipkan ringkasan sebelumnya utuh, bersarang tanpa batas.
Konsolidasi otomatis	Fitur kerangka kerja yang menulis ulang memori agen di malam hari tanpa manusia.
Penghubung (MCP)	Protokol standar untuk menyambungkan tool eksternal (basis data, penyimpanan) ke model.
Token	Satuan teks yang dihitung penyedia model; kira-kira tiga perempat kata.

LAMPIRAN B. Perbandingan ruang kerja agen dan chat engine

Peran	Ruang kerja agen	Chat engine
Aturan perilaku yang selalu berlaku	berkas perilaku di akar ruang kerja	identitas, karakter, persona
Indeks penunjuk	daftar pointer satu baris per rujukan	indeks kemampuan
Prosedur rinci, dibaca saat dipicu	berkas di direktori referensi	kontrak per maksud
Pembeda kasus yang mudah tertukar	catatan jebakan pada berkas terkait	berkas pembeda per maksud
Pemicu pembacaan	kata kerja perintah pada pointer	maksud terpilih hasil pengenalan

LAMPIRAN C. Garis waktu

Tanggal	Kejadian di lapangan	Provenance	Literatur sejamin
24 Mei 2026	Tabel ringkasan lahir (bom dipasang)	P18	MMPO 28 Mei
29 Mei	Lumbung catatan lahir; ADR ke-12	P1	Karpathy LLM Wiki, Apr
26–27 Jun	Fallback runtime, amnesia, migrasi, aturan bebas vendor	P4, P7, P8	
29–30 Jun	Chat engine tipis di depan; batas riwayat 12 jadi 3	P32, P22	
6 Jul	Dua daemon Docker di staging; pengujian mengenai stack lama	P37	
14 Jul	Kunci nyasar di papan staging; RCA matryoshka	P10–P16, P20	
14–15 Jul	Ringkasan dicabut total	P23	
22 Jul	Panel: agen 42–56 ribu per giliran	P30, P31	
24 Jul	Audit memori; konsolidasi otomatis dibasmi; prinsip frekuensi akses	P26, P6	Anthropic “new rules”, 24 Jul
29 Jul	Teguran 100 ribu token; doktrin anti-rampok	P33	
8 Agu	Entri promosi otomatis dihapus, tidak pernah dipanggil	P25	“When to Forget”, Apr
12 Agu	Panel: chat engine 3,3–4,6 ribu, datar	P29	
22 Agu	Dokumen arsitektur memori ditulis	P3	
2 Sep	Meteran token diperbaiki	P34	
3 Sep	Artefak konsolidasi dipulihkan, diverifikasi dua pihak	P24	
8 Sep	Penghubung bocor, enam dimatikan	P17	OKF Agent Memory, Sep
10 Sep	2.860 arsip terhapus	P27	
15 Sep	Sembilan tagihan artefak	P2	

LAMPIRAN D. Provenance

Semua artefak di bawah berada di repositori privat operator. Reviewer yang diberi akses baca dapat memverifikasi tiap baris. Hash commit disingkat 7 karakter. Nama berkas dan kunci ditulis apa adanya di sini, tidak di badan teks.

ID	Tanggal	Artefak	Jenis
P1	2026-05-29 08:19	Lambung catatan, commit 022c5da (README) dan a4341c4 (AGENTS.md)	riwayat versi
P2	2026-09-15	Sembilan berkas setoran tagihan, disimpan operator	berkas setoran
P3	2026-08-22 08:03	Lambung catatan, 03-ARCHITECTURE/agent-memory-architecture.md, commit f4c5598	riwayat versi
P4	2026-06-26 23:46	Ruang kerja agen, commit 852f8d7 (migrasi memori tahap 2; berkas terkursi 125→88 baris)	riwayat versi
P5	2026-07-14 10:14-10:23	Arsip percakapan agen deploy, topik builder; penempatan aturan routing	arsip percakapan
P6	2026-07-24	Catatan harian agen QA, workspace-kuli-qa/memory/2026-07-24.md	catatan harian
P7	2026-06-26 23:34	Ruang kerja agen, commit 42433af (migrasi memori tahap 1, 19 berkas)	riwayat versi
P8	2026-06-27 05:53	Ruang kerja agen, commit 2219245 (workspace-shared/feedback_vendor_neutral_memory.md)	riwayat versi
P9	2026-06-27 pagi	Arsip percakapan agen deploy; kalimat operator “auto amnesia auto ngawur”	arsip percakapan
P10	2026-07-14 10:11	Arsip percakapan agen deploy; operator menemukan ctx:build-chat_engine di Redis staging	arsip percakapan
P11	2026-07-15 11:46	Ruang kerja agen, commit fa568e8 (feedback_redis_routing.md, lahir 14 Jul 10:14)	riwayat versi
P12	2026-07-14 10:41	Ruang kerja agen, commit 91fe71c (“hapus semua mac reference dari memory”)	riwayat versi
P13	2026-06-30 / 07-03	workspace-shared/project_builder_topic_defaults.md, commit 0e3d0de, 05aa791: “rpush ... db=7” tanpa host	riwayat versi
P14	2026-07-02	Trajectory agen deploy: tool redis__rpush dan mac-redis__rpush di context	arsip percakapan
P15	2026-06-21	~/openclaw/mcp-servers/redis-extended.mjs: DB default dari DSN, deskripsi parameter hardcoded “default: 7”	kode
P16	2026-07-10 (masuk riwayat)	workspace/memory/feedback_auto_ctx_log.md, commit 58cd841; dihapus 91fe71c	riwayat versi
P17	2026-09-08 18:30-19:51	Arsip percakapan agen operasi, grup DevOps; backup config openclaw.json sebelum/sesudah	arsip percakapan
P18	2026-05-24	Chat engine, commit 8634dbb, fb09101 (tabel ai_memory_summary, fungsi build_memory_summary)	riwayat versi
P19	2026-05-26	Basis data uji chat engine, satu baris ringkasan bersarang dua lapis	basis data uji
P20	2026-07-14 18:03	Chat engine, tiket V2.BUG-5, commit 78e1293	tiket
P21	2026-07-14	Catatan harian agen QA, dipulihkan dari tempat sampah 15 Sep; sha256 5df37415...	catatan harian
P22	2026-06-30 08:33	Chat engine, tiket RT-4, commit a038531	riwayat versi
P23	2026-07-14/15	Chat engine, commit 88133b5, 8a3dd10; migrasi 20260714_0009	riwayat versi
P24	2026-06-16 s/d 07-14	Artefak dreaming (DREAMS.md), dipulihkan dari tempat sampah 3 Sep; verifikasi dua pihak	artefak
P25	2026-08-08	Catatan harian agen QA, 2026-08-08.md, penghapusan entri TC-3	catatan harian
P26	2026-07-24 19:25	Ruang kerja agen, commit 878fd58 (29 berkas), stempel waktu tempat sampah	riwayat versi
P27	2026-09-10	Catatan harian agen operasi; aturan “jangan	catatan harian

		hapus jsonl tanpa ketok"	
P28	2026-07-24 19:25:18	Tempat sampah OS, 29 berkas catatan harian agen QA dengan .trashinfo	tempat sampah
P29	2026-08-12	Tangkapan layar panel pemakaian penyedia, 19 baris deepseek-v4-pro	tangkapan layar
P30	2026-07-20/22	Tangkapan layar panel pemakaian penyedia, baris glm-5.2 42-56 ribu	tangkapan layar
P31	2026-07-22 17:19	Record runtime agen riset (kuli-oriental), input 42.468 output 76	arsip runtime
P32	2026-06-30	Chat engine, commit e52e489 (klasifikasi hanya membaca dua berkas), RT-1 29 Jun (lazy load)	riwayat versi
P33	2026-07-29 19:41	Arsip percakapan agen riset; teguran "100k token, mau rampog lu"	arsip percakapan
P34	2026-09-02	Chat engine, tiket V2.BUG-18, commit 8bdf2e5 (stream_usage=True)	riwayat versi
P35	2026-08-19	Dokumen "Arsitektur Konteks pada Chat Engine", disusun agen deploy	dokumen
P36	2026-09-15	Setoran tagihan 1, bagian B4: dispatcher default 12, graph 3	berkas setoran
P37	2026-07-06 17:58-19:33	Arsip kanal kerja bersama; dua daemon Docker, pengujian mengenai stack lama	arsip percakapan

LAMPIRAN E. Rujukan

[1] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.

[2] Park, J. S., et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. UIST 2023.

[3] Lin, K., Snell, C., Wang, Y., Packer, C., Wooders, S., Stoica, I., & Gonzalez, J. E. (2025). Sleep-time Compute: Beyond Inference Scaling at Test-time. arXiv:2504.13171.

[4] Hong, K., Troynikov, A., & Huber, J. (2025). Context Rot: How Increasing Input Tokens Impacts LLM Performance. Chroma technical report, July 2025.

[5] Anthropic (2025). Effective context engineering for AI agents. Engineering blog, September 2025.

[6] Shihpar, T., Anthropic (2026). The new rules of context engineering for Claude 5 generation models. 24 July 2026.

[7] AGENTS.md specification (2025), Agentic AI Foundation, Linux Foundation; adoption figures per arXiv:2604.21090.

[8] Simsek, B. (2026). When to Forget: A Memory Governance Primitive. arXiv:2604.12007.

[9] Verbatim Chunks Beat Extracted Artifacts: A Controlled Ablation of Memory Representations for Long LLM Conversations (2026). arXiv:2601.00821. Penulis belum diverifikasi.

[10] Karpathy, A. (2026). LLM Wiki (gist), April 2026.

[11] Meta-Cognitive Memory Policy Optimization for Long-Horizon LLM Agents (2026). arXiv:2605.30159. Penulis belum diverifikasi.

Catatan: rujukan [9] dan [11] dibaca dari abstrak; nama penulis akan dilengkapi sebelum versi 1.0.

Authority by upbringing, not by degree.